

ESPRIT School of Business

Master 1 — Business Analytics (M1 BA)

Projet Intégré

Analyse et Prédiction du Churn Client

Analyse et Prédiction du Churn Client

Équipe projet

Israa Guedri, Eya Smeti, Donia Belamin, Loujaina Guesmi

Tuteur du projet

Aymen Ben Brik

Juillet 2026

Rapport final de projet intégré

Table des matières

1	Résumé exécutif	3
2	1. Introduction et contexte	4
2.1	1.1 Contexte métier	4
2.2	1.2 Objectif du projet	4
2.3	1.3 Démarche et organisation	4
2.4	1.4 Structure du présent document	4
3	2. Description et exploration des données	5
3.1	2.1 Origine et volumétrie	5
3.2	2.2 Principales variables	5
3.3	2.3 Définition de la variable cible	5
3.4	2.4 Qualité des données et problèmes identifiés	6
3.5	3.1 Vue d'ensemble	6
3.6	3.2 Justification des choix techniques	6
3.7	3.3 Flux de données de bout en bout	8
4	4. ETL et modélisation dimensionnelle	9
4.1	4.1 Pipeline ETL	9
4.1.1	4.1.1 Extraction (extract.py)	9
4.1.2	4.1.2 Transformation (transform.py)	9
4.1.3	4.1.3 Chargement (load.py)	9
4.2	4.2 Modèle dimensionnel	10
4.3	4.3 Contrôles de cohérence et de qualité	10
4.4	5.1 Démarche	11
4.5	5.2 KPIs retenus	11
4.6	5.3 État d'avancement du rapport Power BI	11
4.7	5.4 Regroupement des catégories de compte	13
4.8	6.1 Objectif et grain de la modélisation	13
4.9	6.2 Préparation des données et fuites de données identifiées	14
4.9.1	6.2.1 Construction du jeu de données	14
4.9.2	6.2.2 Fuites de données identifiées et corrigées	14
4.9.3	6.2.3 Encodage et prétraitement	14
4.9.4	6.2.4 Gestion du déséquilibre de classes	15
4.9.5	6.2.5 Séparation train / test	15
4.10	6.3 Modèles entraînés et métriques d'évaluation	15
4.11	6.4 Modèle retenu	16
4.12	6.5 Rapport de classification détaillé	17
4.13	6.6 Interprétabilité	17
4.14	6.7 Sauvegarde et intégration du modèle	20
4.15	7.1 Objectifs et périmètre fonctionnel	20
4.16	7.2 Choix technique	20
4.17	7.3 Page d'accueil — KPIs de synthèse	20
4.18	7.4 Analyse du churn par segment	20
4.19	7.5 Prédiction individuelle et comptes à risque	20
4.20	7.6 Déploiement	23
5	8. Limites et perspectives	25
5.1	8.1 Limites méthodologiques	25
5.2	8.2 Limites d'ingénierie	25
5.3	8.3 Perspectives d'amélioration	26

6 9. Conclusion	27
7 Annexes	28
7.1 A. Glossaire	28
7.2 B. Structure du dépôt de code	28
7.3 C. Références des outils utilisés	29

1. Résumé exécutif

Ce projet propose une chaîne décisionnelle complète pour comprendre, mesurer et anticiper l'attrition (*churn*) de la clientèle d'une institution bancaire, à partir d'un jeu de données réel et anonymisé de **528 883 lignes** (couples client-compte-produit), portant sur **363 569 clients**.

La solution mise en œuvre couvre les cinq composantes attendues : un pipeline ETL reproductible qui nettoie et enrichit les données brutes ; un entrepôt de données PostgreSQL modélisé en étoile ; un rapport Power BI d'analyse descriptive du churn ; un modèle de Machine Learning (XGBoost) qui prédit la probabilité de churn d'un client avec un **PR-AUC de 0,983** ; et une application web Streamlit qui expose ce modèle et ces analyses à un utilisateur métier.

Le taux de churn observé au niveau client (tous comptes clôturés) est de **44,5 %**. L'analyse a mis en évidence un facteur protecteur particulièrement net : les clients détenant un produit d'épargne ou de placement affichent un taux de churn de l'ordre de 8 % contre plus de 40 % pour les autres — un résultat convergent, retrouvé à la fois dans l'analyse descriptive (Power BI, SQL) et dans l'interprétabilité du modèle prédictif (SHAP).

Une attention particulière a été portée à la rigueur méthodologique : deux fuites de données ont été identifiées et corrigées en cours de modélisation (une variable définissant littéralement la cible, une autre corrélée à un artefact de complétude du système source plutôt qu'à un vrai comportement client), documentées avec un test de sensibilité chiffré plutôt que simplement écartées sans justification.

L'ensemble du code est disponible sur un dépôt GitHub public, structuré en composantes numérotées, chacune dotée d'un README détaillé permettant la reproduction complète du pipeline par un tiers.

2. 1. Introduction et contexte

2.1 1.1 Contexte métier

La fidélisation client est un enjeu majeur du secteur bancaire : la littérature professionnelle du secteur estime qu'acquérir un nouveau client coûte significativement plus cher que d'en conserver un existant. Anticiper le départ d'un client — le *churn* — permet de déclencher des actions de rétention ciblées avant qu'il ne soit trop tard, plutôt que de subir l'attrition de manière passive.

Ce projet s'inscrit dans le cadre du Projet Intégré du Master 1 Business Analytics d'ESPRIT School of Business. Une institution bancaire (dont l'identité n'est pas mentionnée dans ce document, conformément aux règles de confidentialité du projet) a mis à disposition un extrait anonymisé de son système d'information : informations clients, comptes et produits, sur plusieurs années d'historique.

2.2 1.2 Objectif du projet

L'objectif est de concevoir, de bout en bout, une solution analytique complète permettant de :

1. **Comprendre** le churn : quels segments de clientèle sont les plus exposés, quels facteurs y sont associés.
2. **Mesurer** le churn : le quantifier de façon fiable et le restituer via des indicateurs métier exploitables.
3. **Anticiper** le churn : prédire, pour un client donné, sa probabilité de churn à l'aide d'un modèle supervisé.
4. **Restituer** ces analyses à des utilisateurs non techniques via des tableaux de bord et une application web.

2.3 1.3 Démarche et organisation

Le projet a été mené sur 4 semaines, avec une progression suivant la chaîne de valeur analytique classique : exploration des données, ETL, modélisation dimensionnelle, Business Intelligence, Machine Learning, puis interface de restitution. Cette organisation correspond à la structure numérotée du dépôt de code (01_etl à 07_presentation).

Le travail a été mené en équipe de 4, avec une répartition des rôles centrée sur les compétences de chacune : ingénierie des données et Machine Learning en binôme, Business Intelligence, et structuration des livrables finaux (rapport, présentation). Le détail de la répartition figure dans le README principal du dépôt.

2.4 1.4 Structure du présent document

Ce rapport suit la structure recommandée par le guide du projet : après cette introduction, la section 2 décrit et explore les données sources. La section 3 présente l'architecture globale de la solution. La section 4 détaille le pipeline ETL et la modélisation dimensionnelle. La section 5 présente les analyses Business Intelligence et le rapport Power BI. La section 6 constitue le cœur méthodologique du projet : la modélisation Machine Learning, de la préparation des données à l'interprétabilité du modèle final. La section 7 présente l'application web et son déploiement. La section 8 discute des limites du travail réalisé et des perspectives d'amélioration, avant la conclusion.

3. 2. Description et exploration des données

3.1 2.1 Origine et volumétrie

Les données mises à disposition proviennent du système d'information d'une institution bancaire et ont été anonymisées avant remise aux équipes projet : aucun identifiant personnel direct n'est exploitable (les identifiants clients et comptes ont été remplacés par des séquences artificielles), mais la structure et la cohérence métier des données ont été préservées.

Le jeu de données est composé de :

- **Un fichier principal** au format CSV, contenant les informations clients, comptes et produits, avant nettoyage.
- **Huit tables de dimensions** au format Excel, donnant le libellé descriptif associé aux codes présents dans le fichier principal (catégorie de compte, devise, motif de clôture, secteur d'activité, segment cible, secteur détaillé, référentiel transaction, référentiel agence).

Chaque ligne du fichier principal correspond à un couple **(client, compte-produit)** : un même client peut apparaître plusieurs fois s'il détient plusieurs comptes ou produits.

Indicateur	Valeur
Lignes brutes (avant nettoyage)	528 883
Doublons identifiés et supprimés	38 640
Lignes finales (table de faits)	490 243
Clients uniques	363 569
Colonnes du fichier final (après enrichissement)	36

3.2 2.2 Principales variables

Le fichier principal regroupe quatre familles de variables :

Identifiants — CUSTOMER_NO, ACCOUNT_NO, BRANCH (agence de rattachement).

Informations client — nationalité, résidence, statut marital, année de naissance, nature du client (personne physique / morale), classification commerciale (PARTYCLASS : Retail, Corporate...), ligne métier, score KYC (*Know Your Customer*), complétude du dossier, secteur d'activité, salaire déclaré.

Informations compte — statut du compte (ACCOUNT_STATUS, *Active/Closed* — base de la variable cible), dates d'ouverture et de clôture, catégorie de compte, devise, motif de clôture, solde.

Informations produit — groupe de produits, ligne de produits, produit spécifique.

3.3 2.3 Définition de la variable cible

Deux niveaux de churn ont été retenus, cohérents avec le grain différent de l'analyse descriptive (niveau compte-produit) et de la modélisation prédictive (niveau client) :

- **CHURN** (niveau compte-produit) : 1 si le compte est au statut *Closed*, 0 sinon. Utilisé pour les analyses de répartition par produit, secteur, devise.

- **CLIENT_FULL_CHURN** (niveau client) : 1 si **tous** les comptes du client sont clôturés (`NB_COMPTE_CLOS = NB_COMPTE`), 0 sinon. C'est un signal plus fort qu'un churn partiel d'un client multi-comptes, et c'est la variable retenue comme cible du modèle prédictif (section 6).

Sur le jeu de données final : **37,4 %** des lignes compte-produit sont en churn, et **44,5 %** des clients sont en churn complet — un déséquilibre de classes modéré mais réel, qui a orienté le choix des métriques d'évaluation du modèle (section 6.3).

3.4 2.4 Qualité des données et problèmes identifiés

L'exploration initiale a mis en évidence plusieurs problèmes de qualité, traités explicitement dans le pipeline ETL (détaillé section 4) :

- **38 640 doublons** dans le fichier source, supprimés avant chargement.
- **Dates aberrantes** : certaines dates d'ouverture de compte antérieures à 1900 (vraisemblablement des valeurs de remplissage technique plutôt que de vraies dates), filtrées via un seuil `MIN_DATE = 1900-01-01`.
- **Incohérences de clôture** : des comptes marqués comme clôturés sans motif de clôture renseigné, ou l'inverse — un indicateur `FLAG_INCOHERENCE_CLOTURE` a été créé pour tracer ce phénomène sans le masquer.
- **Valeurs catégorielles manquantes non uniformisées** : certains champs texte utilisaient des valeurs de remplissage variées (chaînes vides, codes UNKNOWN, valeurs NULL textuelles) selon la colonne, uniformisées lors de la transformation.
- **Une part significative de comptes sans catégorie de compte renseignée** (`ACCOUNT_CATEGORY_LABEL = "Non disponible"`, ~55 % des lignes) — un constat rapporté tel quel plutôt que masqué, avec ses implications discutées section 8 (limites).

Ces choix de traitement sont documentés en détail dans le code source du module de transformation (`01_etl/src/transform.py`) et dans le dictionnaire de données du dépôt.
3. Architecture de la solution

3.5 3.1 Vue d'ensemble

La solution repose sur une chaîne de traitement linéaire, où chaque composante consomme la sortie de la précédente : les données brutes sont nettoyées et enrichies par un pipeline ETL, chargées dans un entrepôt de données PostgreSQL modélisé en étoile, puis exploitées en parallèle par deux voies de restitution — l'analyse descriptive via Power BI, et la modélisation prédictive via un pipeline Machine Learning dont le modèle final est exposé par une application web.

Un choix structurant du projet a été de faire de **PostgreSQL la source de vérité unique** pour l'ensemble des composantes en aval (Power BI, Machine Learning, application web) plutôt que de dupliquer des exports CSV intermédiaires. Concrètement, `04_machine_learning/src/prepare_data.py` construit le jeu de données d'entraînement par une requête SQL d'agrégation directe sur l'entrepôt (pas de fichier CSV intermédiaire caché), et `05_web_app` interroge ce même entrepôt en temps réel à chaque affichage de page. Ce choix garantit qu'il n'existe qu'une seule définition du churn et qu'un seul jeu de features dans tout le projet.

3.6 3.2 Justification des choix techniques



Fig. 1 : Architecture globale de la solution

Composante	Choix retenu	Justification
Langage principal	Python 3.13	Écosystème data science complet, cohérence entre ETL, ML et application web
Entrepôt de données	PostgreSQL	Base relationnelle mature, requêtable directement par Power BI et par le pipeline ML, plus proche d'un contexte de production qu'une base fichier
Manipulation de données	pandas	Standard de facto, suffisant pour la volumétrie du projet (< 1 million de lignes)
Modélisation dimensionnelle	Schéma en étoile	Grain compte-produit, dimensions dénormalisées — optimisé pour les requêtes analytiques de Power BI et les agrégations SQL du pipeline ML
Visualisation BI	Power BI Desktop	Imposé par l'énoncé du projet
Modélisation ML	scikit-learn + XGBoost	Couverture des familles de modèles classiques et boostés, API cohérente (pipelines scikit-learn)
Interface web	Streamlit	Développement rapide en Python pur, cohérent avec le reste de la stack, sans changement de langage pour l'équipe

Composante	Choix retenu	Justification
Versioning	Git + GitHub	Imposé par l'énoncé du projet

3.7 3.3 Flux de données de bout en bout

1. **Extraction** : lecture du fichier CSV principal et des tables de dimensions Excel.
2. **Transformation** : nettoyage, déduplication, jointures avec les dimensions, calcul de variables dérivées (âge, ancienneté, indicateurs de qualité).
3. **Chargement** : écriture dans les 7 tables du schéma en étoile de l'entrepôt PostgreSQL churn_dw.
4. **Consommation BI** : Power BI se connecte directement à l'entrepôt et calcule ses mesures en DAX.
5. **Consommation ML** : le pipeline de préparation de données interroge l'entrepôt par une requête SQL d'agrégation (grain client), entraîne et évalue plusieurs modèles, puis sérialise le modèle final retenu.
6. **Consommation applicative** : l'application web interroge l'entrepôt pour les KPIs et les listes de clients, et charge le modèle sérialisé pour la prédiction.

4. 4. ETL et modélisation dimensionnelle

4.1 4.1 Pipeline ETL

Le pipeline ETL est implémenté en Python (pandas), organisé en trois modules distincts correspondant aux trois étapes classiques Extract-Transform-Load, orchestrés par un script principal (01_etl/pipeline.py) qui peut s'exécuter sur l'intégralité des données ou sur un échantillon (paramètre `--sample`, utile pour les tests rapides en développement).

4.1.1 4.1.1 Extraction (extract.py)

Cette étape charge le fichier CSV principal ainsi que les quatre tables de dimensions Excel utilisées par le pipeline (catégories de compte, devises, motifs de clôture, secteurs d'activité). Chaque chargement est accompagné d'un contrôle de cohérence (nombre de lignes chargées, colonnes présentes) journalisé en sortie.

4.1.2 4.1.2 Transformation (transform.py)

C'est l'étape la plus substantielle du pipeline. Elle réalise, dans l'ordre :

- **Déduplication** : suppression des lignes strictement dupliquées (38 640 lignes supprimées sur les 528 883 lignes source).
- **Normalisation des dates** : conversion des dates au format YYYYMMDD en type `datetime`, avec filtrage des dates aberrantes antérieures au 1er janvier 1900 (`MIN_DATE`), traitées comme des valeurs de remplissage technique plutôt que des dates réelles.
- **Jointures avec les dimensions** : enrichissement du fichier principal avec les libellés descriptifs (secteur d'activité, devise, motif de clôture, catégorie de compte) à partir des codes présents dans les données sources.
- **Calcul de variables dérivées** :
 - AGE à partir de l'année de naissance.
 - CLIENT_SENIORITY_YEARS et ACCOUNT_SENIORITY_YEARS (ancienneté en années, calculée par rapport à une date de référence).
 - DAYS_SINCE_LAST_REVIEW (fraîcheur du dossier KYC).
 - NB_COMPTEs, NB_PRODUITS_DISTINCTS, NB_COMPTEs_CLOS (agrégats de multi-bancarisation par client).
 - CHURN et CLIENT_FULL_CHURN (variables cibles, voir section 2.3).
 - FLAG_INCOHERENCE_CLOTURE (indicateur de qualité de données, sans masquer l'anomalie détectée).
- **Contrôle de cohérence final** : vérification du nombre de lignes, du taux de valeurs manquantes par colonne, et du taux de churn obtenu, journalisés en sortie du pipeline.

4.1.3 4.1.3 Chargement (load.py)

Cette étape construit les 7 tables du schéma en étoile à partir du DataFrame transformé (génération des clés de substitution — *surrogate keys* — pour chaque dimension), puis les charge dans PostgreSQL. Pour la table de faits (490 243 lignes), le chargement utilise la commande `COPY` de PostgreSQL plutôt que des insertions ligne par ligne, ce qui réduit le temps de chargement complet à quelques dizaines de secondes contre plusieurs minutes avec une approche par insertions unitaires.

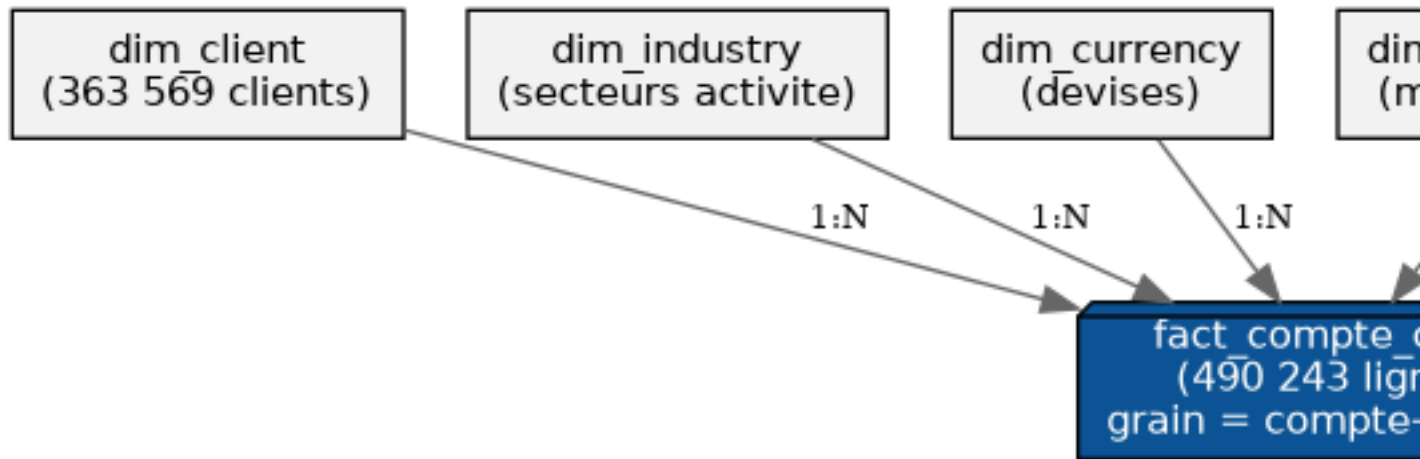


Fig. 2 : Schéma en étoile de l'entrepôt churn_dw

4.2 4.2 Modèle dimensionnel

Le schéma retenu est un schéma en étoile classique : une table de faits centrale, `fact_compte_client`, entourée de six tables de dimensions.

Grain de la table de faits : un couple (client, compte-produit) — cohérent avec le grain du fichier source, et suffisamment fin pour permettre à la fois les analyses au niveau compte (Power BI) et l'agrégation au niveau client (Machine Learning) sans perte d'information.

Table	Rôle	Volumétrie
<code>fact_compte_client</code>	Table de faits — 1 ligne = 1 compte-produit	490 243 lignes
<code>dim_client</code>	Attributs socio-démographiques du client	363 569 lignes
<code>dim_industry</code>	Secteur d'activité du client	644 lignes
<code>dim_currency</code>	Devise du compte	19 lignes
<code>dim_closure_reason</code>	Motif de clôture du compte	19 lignes
<code>dim_account_category</code>	Catégorie du compte	149 lignes
<code>dim_date</code>	Dimension calendaire (dates d'ouverture, de clôture)	13 070 lignes

Les scripts SQL de création du schéma (`02_data_warehouse/schema/create_tables.sql`) définissent les clés primaires, contraintes de clé étrangère, et index sur les colonnes de jointure les plus fréquemment utilisées (clés de substitution, `CUSTOMER_NO`, `ACCOUNT_NO`, indicateur `CHURN`).

4.3 4.3 Contrôles de cohérence et de qualité

Au-delà des traitements de nettoyage décrits en 4.1.2, le pipeline documente explicitement les problèmes de qualité résiduels plutôt que de les masquer :

- Le taux de churn obtenu après transformation (37,4 % au niveau compte, 44,5 % au niveau client) est vérifié à chaque exécution du pipeline et journalisé, pour détecter toute dérive anormale entre deux exécutions.
- L'indicateur FLAG_INCOHERENCE_CLOTURE trace les comptes présentant une incohérence entre statut et motif de clôture, sans les exclure du jeu de données — un choix délibéré pour ne pas biaiser la volumétrie, la correction de fond de ces incohérences relevant du système source et non du pipeline analytique.
- La proportion de comptes sans catégorie renseignée (Non disponible, ~55 % des lignes) est également documentée telle quelle, avec les implications de ce constat discutées section 8. # 5. Analyses BI et tableaux de bord Power BI

4.4 5.1 Démarche

Le rapport Power BI se connecte à l'entrepôt churn_dw (dans sa version la plus récente, via un export CSV des 7 tables de l'entrepôt afin de simplifier la configuration du poste de travail, sans changer ni le contenu ni la définition des indicateurs par rapport à une connexion PostgreSQL directe — voir 03_power_bi/README.md). Les relations entre la table de faits et les dimensions ont été reconstruites dans Power BI selon le même schéma en étoile que celui de l'entrepôt (section 4.2).

L'ensemble des mesures DAX utilisées est documenté dans 03_power_bi/measures_dax.md, avec pour chacune une explication de sa logique métier plutôt qu'une simple formule brute — un choix qui facilite la relecture et la maintenance du rapport par un tiers.

4.5 5.2 KPIs retenus

Les indicateurs suivants ont été définis, cohérents avec ceux utilisés par ailleurs dans le projet (entrepôt SQL, modélisation ML) :

- **Taux de churn compte** : proportion de lignes compte-produit dont le compte est clôturé (37,4 % sur l'ensemble du portefeuille).
- **Taux de churn client (full churn)** : proportion de clients dont tous les comptes sont clôturés (44,5 %) — l'indicateur le plus représentatif d'une perte de relation client complète.
- **Répartition par secteur d'activité** : le taux de churn varie fortement d'un secteur à l'autre, certains secteurs peu représentés atteignant des taux proches de 100 % (à interpréter avec prudence compte tenu de leur faible effectif, voir section 8).
- **Répartition des motifs de clôture** : plus de 70 % des clôtures sont associées à un motif générique (« Autre ») ou non renseigné, ce qui limite la capacité d'analyse fine des causes de churn côté système source.
- **Évolution mensuelle des clôtures** : mise en évidence de pics ponctuels, à confronter à des événements métier (campagnes, changements de politique commerciale) que les données à disposition ne permettent pas d'identifier avec certitude.

4.6 5.3 État d'avancement du rapport Power BI

À la date de rédaction de ce rapport, une page opérationnelle a été construite, combinant les indicateurs clés (cartes de synthèse), la répartition du churn par secteur d'activité, la répartition des motifs de clôture et l'évolution temporelle des clôtures :

Conformément au guide du projet, deux axes d'analyse complémentaires restent à finaliser dans une page dédiée à la segmentation client (profils, valeur client, comptes

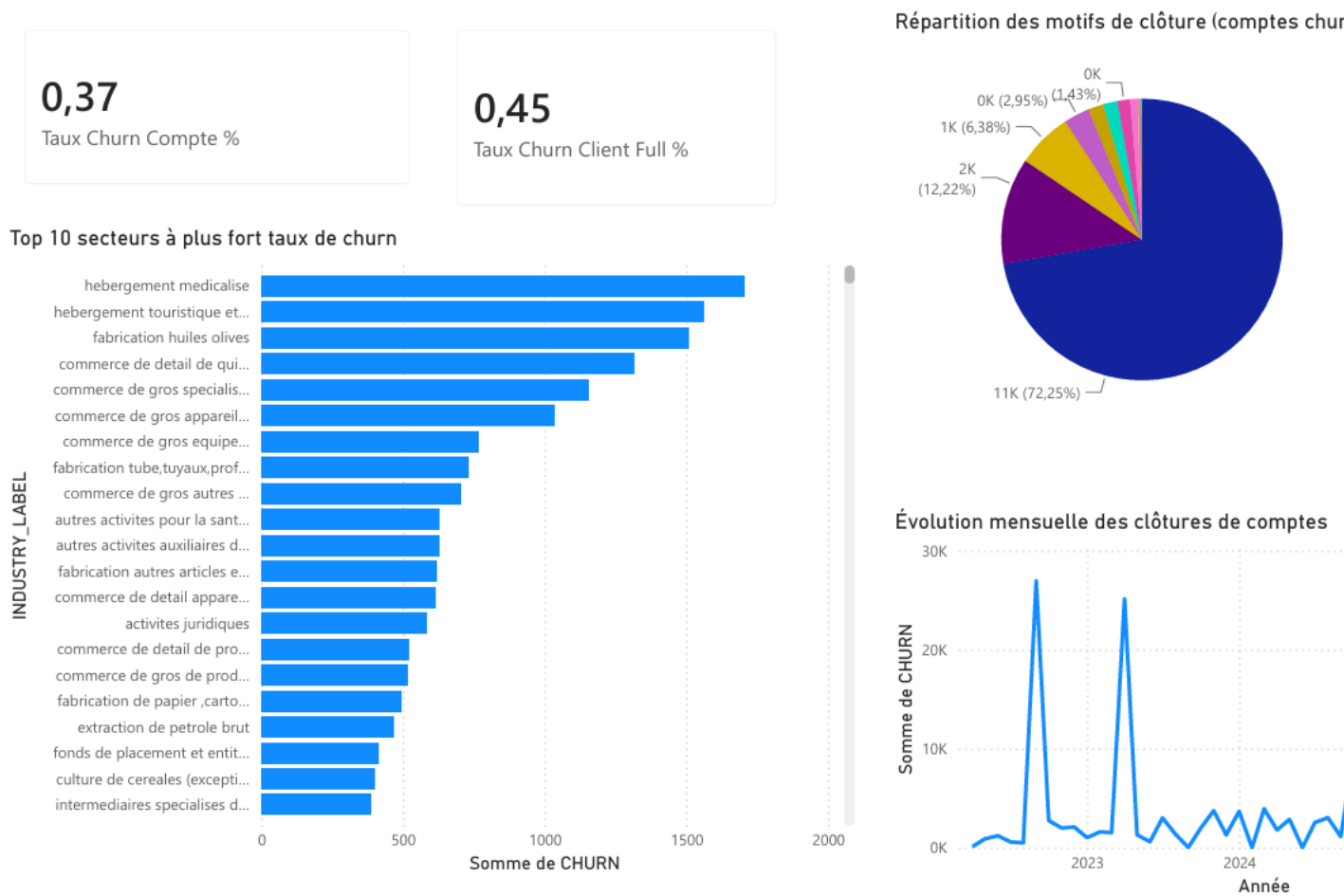


Fig. 3 : Page Power BI — Vue d'ensemble et analyse du churn

à risque) — la structure de cette page ainsi que les mesures DAX nécessaires sont d'ores et déjà documentées et prêtes à l'implémentation (03_power_bi/measures_dax.md, bloc 5 : détention de produits, bloc 3 : regroupement des catégories de compte). Le tableau de correspondance mesure/visuel du même document sert de feuille de route pour cette finalisation.

4.7 5.4 Regroupement des catégories de compte

Un travail spécifique a été mené pour rendre exploitable la variable `ACCOUNT_CATEGORY_LABEL`, qui compte près de 150 modalités distinctes dans les données sources — dont plus de la moitié sans libellé renseigné (« Non disponible ») et le reste réparti en de très nombreuses catégories très fines (ex. « CPTÉ SOUS DELAG.DE CHANGE », quelques dizaines de comptes). Un tel niveau de détail rend toute visualisation illisible.

Ces catégories ont été regroupées en 7 familles métier (colonne calculée `Famille Compte`, cf. 03_power_bi/measures_dax.md), avec un résultat particulièrement parlant :

Famille	Part du portefeuille	Taux de churn
Non renseigné	54,9 %	58,2 %
Épargne	36,5 %	8,6 %
Compte courant / dépôt à vue	6,8 %	24,1 %
Allocation touristique / change	0,7 %	40,6 %
Compte indisponible / bloqué	0,6 %	40,9 %
Compte professionnel	0,3 %	47,8 %
Autre	0,2 %	21,0 %

Ce résultat est l'un des constats les plus robustes du projet : la détention d'un produit d'épargne ou de placement est associée à un taux de churn nettement inférieur à toutes les autres familles de comptes. Ce même signal ressort indépendamment de l'analyse d'interprétabilité du modèle Machine Learning (section 6.6), ce qui renforce la confiance dans ce constat — il n'est pas un artefact d'une seule méthode d'analyse.

À l'inverse, la part très importante de comptes « Non renseigné » (54,9 % du portefeuille) est davantage un indicateur de qualité de données côté système source qu'un vrai signal métier ; son taux de churn élevé (58,2 %) doit être interprété avec cette réserve plutôt que comme une catégorie de risque en tant que telle (approfondi section 8). # 6. Modélisation Machine Learning

4.8 6.1 Objectif et grain de la modélisation

L'objectif est de prédire, pour un client donné, sa probabilité de churn complet (`CLIENT_FULL_CHURN`, voir section 2.3). Contrairement aux analyses BI menées au grain compte-produit, la modélisation prédictive se place au **grain client** : chaque ligne du jeu de données d'entraînement représente un client, ses features résultant d'une agrégation SQL de l'ensemble de ses comptes et produits.

Ce choix de grain a une conséquence méthodologique importante, développée en 6.2 : plusieurs variables candidates, naturellement disponibles au niveau compte, deviennent des risques de fuite de données une fois agrégées au niveau client, car elles peuvent se retrouver mécaniquement corrélées — voire identiques — à la définition de la cible.

4.9 6.2 Préparation des données et fuites de données identifiées

4.9.1 6.2.1 Construction du jeu de données

Le jeu de données d'entraînement est construit par une requête SQL unique (04_machine_learning/ qui agrège directement les données de l'entrepôt PostgreSQL par client (GROUP BY CUSTOMER_NO), sans étape intermédiaire de fichier CSV caché. Cette requête produit **363 569 lignes** (une par client), avec des features couvrant :

- Le profil socio-démographique (âge, nationalité, résidence, statut marital, secteur d'activité principal).
- Le comportement bancaire (nombre de comptes, nombre de produits distincts, ancienneté).
- Les indicateurs financiers (solde total et moyen, salaire déclaré, montant total des produits).
- La détention de types de produits (crédit, épargne/placement, compte courant, coffre).

4.9.2 6.2.2 Fuites de données identifiées et corrigées

Un travail de vigilance méthodologique a permis d'identifier deux fuites de données au cours du premier essai d'entraînement, documentées et corrigées plutôt que masquées :

Fuite directe — NB_COMPTES_CLOS. Cette variable a été initialement incluse comme feature candidate. Or CLIENT_FULL_CHURN est *défini* comme $\text{NB_COMPTES_CLOS} = \text{NB_COMPTES}$: la conserver revient à donner au modèle la réponse dans l'énoncé. Le premier essai d'entraînement avec cette variable a donné un score parfait sur toutes les métriques (précision, rappel, F1, AUC = 1,000), un signal sans ambiguïté de fuite de données. La variable a été retirée.

Fuite indirecte — NB_DEVISES. Le nombre de devises distinctes détenues par le client est apparu comme la variable la plus prédictive lors d'un deuxième essai, avec une importance disproportionnée (SHAP). Une investigation a montré que $\text{NB_DEVISES} = 0$ coïncidait avec $\text{CHURN} = 1$ dans près de 100 % des cas pour un sous-groupe de 139 000 clients (38 % du jeu de données) — non pas parce que l'absence de multi-devise cause le churn, mais parce que le champ CURRENCY n'est renseigné dans le système source que lorsque les données produites sont complètes, ce qui coïncide structurellement avec la clôture du compte plutôt que de refléter un comportement client réel. Un **test de sensibilité** a permis de trancher : un modèle XGBoost entraîné sans cette variable obtient un PR-AUC quasi identique (0,9828 contre 0,983 avec), la variable A_EPARGNE_PLACEMENT et MONTANT_TOTAL prenant alors le relais dans l'importance des features — un signal bien plus défendable d'un point de vue métier. La variable a été retirée définitivement.

Cette démarche — détecter, investiguer par un test chiffré, documenter, corriger — est jugée plus rigoureuse qu'un simple retrait a priori de variables suspectes sans vérification, et plus honnête qu'une conservation non questionnée de variables donnant d'excellents résultats sans en comprendre la cause.

4.9.3 6.2.3 Encodage et prétraitement

Le prétraitement est encapsulé dans un pipeline scikit-learn (ColumnTransformer), garantissant que les mêmes transformations sont appliquées de façon identique à l'entraînement et à l'inférence (application web) :

- **Variables numériques** : imputation des valeurs manquantes par la médiane, puis standardisation (StandardScaler).
- **Variables catégorielles** : imputation par une modalité "Inconnu", puis encodage one-hot (OneHotEncoder, avec gestion des modalités inconnues en inférence).
- **Modalités rares** : les modalités peu fréquentes de nationalité, résidence et secteur d'activité sont regroupées sous une catégorie "Autre" avant encodage, pour limiter la dimensionnalité introduite par le one-hot encoding.

4.9.4 6.2.4 Gestion du déséquilibre de classes

Le taux de churn client (44,5 %) constitue un déséquilibre modéré plutôt que sévère. Il a été traité par pondération des classes (`class_weight="balanced"`) plutôt que par un rééchantillonnage synthétique (SMOTE), ce dernier n'apparaissant pas nécessaire compte tenu du déséquilibre limité et présentant un risque de créer des exemples synthétiques peu réalistes sur des variables catégorielles à haute cardinalité.

4.9.5 6.2.5 Séparation train / test

Un découpage stratifié 80 % / 20 % a été retenu (`random_state=42` pour la reproductibilité), soit environ 290 855 clients pour l'entraînement et 72 714 pour l'évaluation.

4.10 6.3 Modèles entraînés et métriques d'évaluation

Six modèles de classification ont été entraînés et comparés, couvrant un spectre de complexité et de familles algorithmiques différentes : régression logistique (référence linéaire), k plus proches voisins, arbre de décision, forêt aléatoire, machine à vecteurs de support (noyau RBF), et XGBoost.

Compte tenu du déséquilibre de classes, l'évaluation ne s'est pas limitée à l'exactitude (*accuracy*), peu informative dans ce contexte, mais a mobilisé un ensemble de métriques adaptées : précision, rappel, F1-score, aire sous la courbe ROC (ROC-AUC) et surtout **aire sous la courbe précision-rappel (PR-AUC)**, la métrique la plus pertinente lorsque la classe positive présente un intérêt métier particulier — ici, identifier les clients à risque de churn.

Limite de comparabilité : les modèles KNN et SVM ne passent pas à l'échelle sur l'intégralité du jeu d'entraînement (coût de prédiction en $O(n_{\text{test}} \times n_{\text{train}})$ pour KNN ; complexité quadratique à cubique en entraînement pour un noyau RBF). Ils ont donc été entraînés sur un sous-échantillon stratifié (respectivement 40 000 et 8 000 lignes), ce qui est indiqué explicitement dans le tableau ci-dessous et doit être gardé à l'esprit dans l'interprétation de leurs résultats.

Modèle	Précision	Rappel	F1	ROC-AUC	PR-AUC	Temps d'entraînement	N entraînement
Régression logistique	0,916	0,893	0,905	0,945	0,949	5,8 s	290 855
Arbre de décision	0,962	0,916	0,938	0,974	0,977	5,2 s	290 855

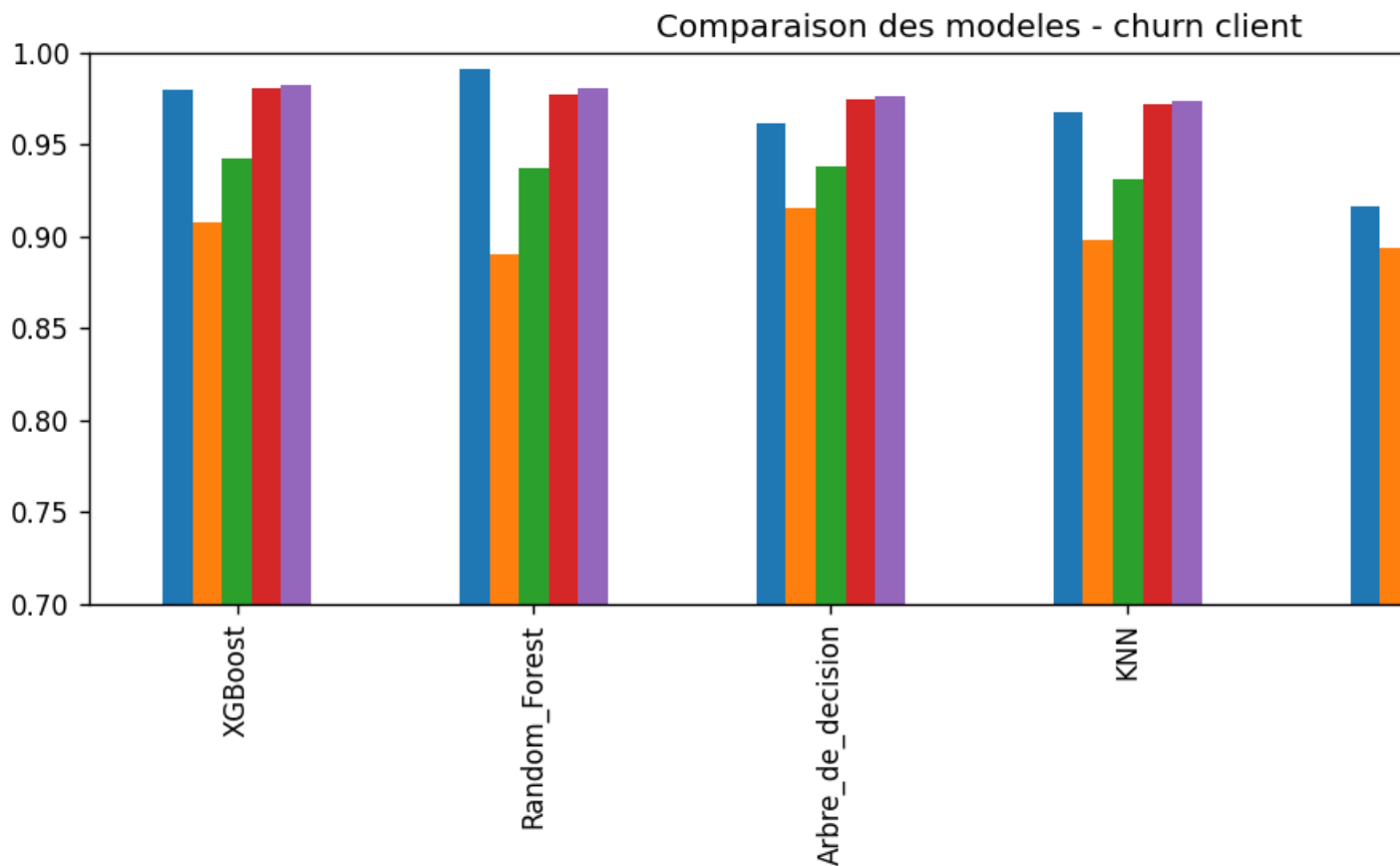


Fig. 4 : Comparaison des 6 modèles sur les cinq métriques d'évaluation

Modèle	Précision	Rappel	F1	ROC-AUC	PR-AUC	Temps d'entraînement	N entraînement
K plus proches voisins	0,967	0,898	0,931	0,972	0,974	7,7 s	40 000*
Forêt aléatoire	0,991	0,890	0,938	0,977	0,981	97,5 s	290 855
SVM (noyau RBF)	0,984	0,871	0,924	0,955	0,948	23,5 s	8 000*
XGBoost	0,980	0,908	0,942	0,981	0,983	15,1 s	290 855

*Sous-échantillon stratifié (voir limite ci-dessus).

4.11 6.4 Modèle retenu

Le modèle **XGBoost** a été retenu comme modèle final, pour les raisons suivantes :

1. **Meilleur PR-AUC (0,983)**, la métrique jugée la plus pertinente compte tenu du

déséquilibre de classes et de l'intérêt métier porté à la classe positive.

2. **Meilleur F1-score (0,942)**, avec un rappel (0,908) élevé sans sacrifier excessivement la précision — un compromis important pour une application de rétention client, où un faux négatif (client qui churn sans avoir été identifié) a un coût métier réel.
3. Entraîné sur l'intégralité des données disponibles, contrairement à KNN et SVM.
4. Temps d'entraînement très raisonnable (15,1 secondes) comparé à la forêt aléatoire (97,5 secondes) pour une performance égale ou supérieure sur l'ensemble des métriques.

La forêt aléatoire obtient une précision légèrement supérieure mais un rappel plus faible, pour un temps d'entraînement environ 6 fois supérieur — le compromis performance/coût de calcul penche nettement en faveur de XGBoost.

4.12 6.5 Rapport de classification détaillé

Sur le jeu de test (72 714 clients) :

	Précision	Rappel	F1-score	Support
Classe 0 (actif)	0,93	0,98	0,96	40 354
Classe 1 (churné)	0,98	0,91	0,94	32 360
Exactitude			0,95	72 714
Moyenne macro	0,95	0,95	0,95	72 714
Moyenne pondérée	0,95	0,95	0,95	72 714

4.13 6.6 Interprétabilité

L'interprétabilité du modèle final a été menée par deux approches complémentaires : l'importance native des features (gain XGBoost) et les valeurs SHAP (*SHapley Additive exPlanations*), calculées sur un échantillon de 2 000 clients du jeu de test.

Les principaux facteurs prédictifs identifiés, une fois les fuites de données de la section 6.2.2 corrigées, sont :

1. MONTANT_TOTAL — montant cumulé des produits détenus par le client. Un montant faible ou nul est associé au churn.
2. ANCIENNETE_COMPTE_MOY_ANNEES — ancienneté moyenne des comptes du client.
3. A_EPARGNE_PLACEMENT — détention d'un produit d'épargne ou de placement, qui ressort comme un facteur **protecteur** contre le churn.
4. NB_COMPTES — nombre de comptes détenus (multi-bancarisation).
5. ANCIENNETE_COMPTE_MAX_ANNEES, ANCIENNETE_CLIENT_ANNEES, SOLDE_MOYEN, SOLDE_TOTAL — ancienneté et niveau de solde, dans l'ordre d'importance décroissante.

Convergence avec l'analyse descriptive. Le facteur protecteur associé à la détention d'un produit d'épargne ou de placement, identifié ici par SHAP, est retrouvé de façon indépendante dans l'analyse Power BI (section 5.4) : les comptes de la famille « Épargne » affichent un taux de churn observé de 8,6 %, contre 24 à 58 % pour les autres familles de comptes. Cette convergence entre deux méthodes d'analyse indépendantes — l'une descriptive et agrégée, l'autre issue de l'interprétabilité d'un modèle prédictif entraîné au grain client — renforce la confiance dans ce constat et en fait une recommandation actionnable prioritaire pour des actions de rétention ciblées.

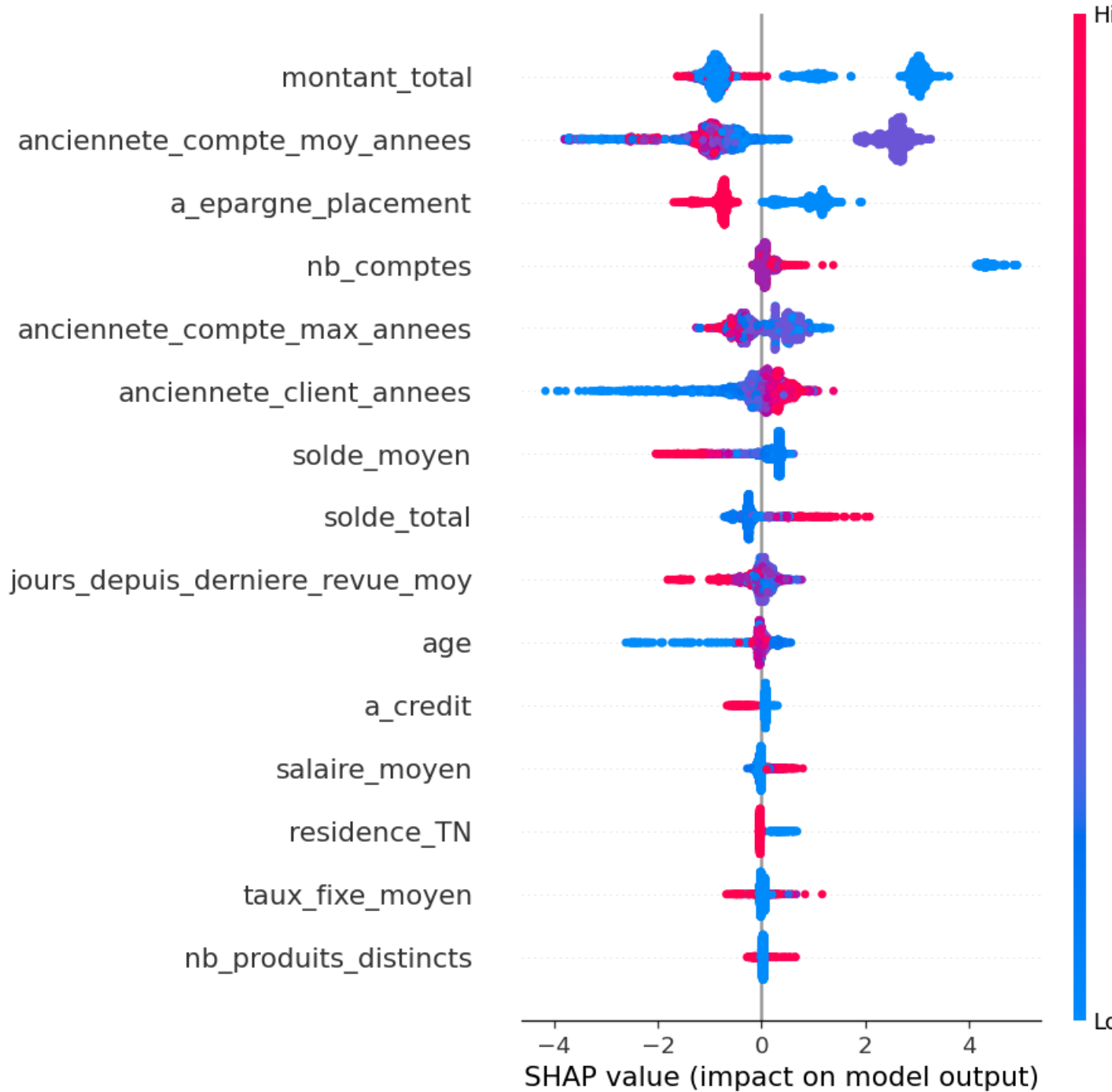


Fig. 5 : Résumé SHAP des features du modèle final

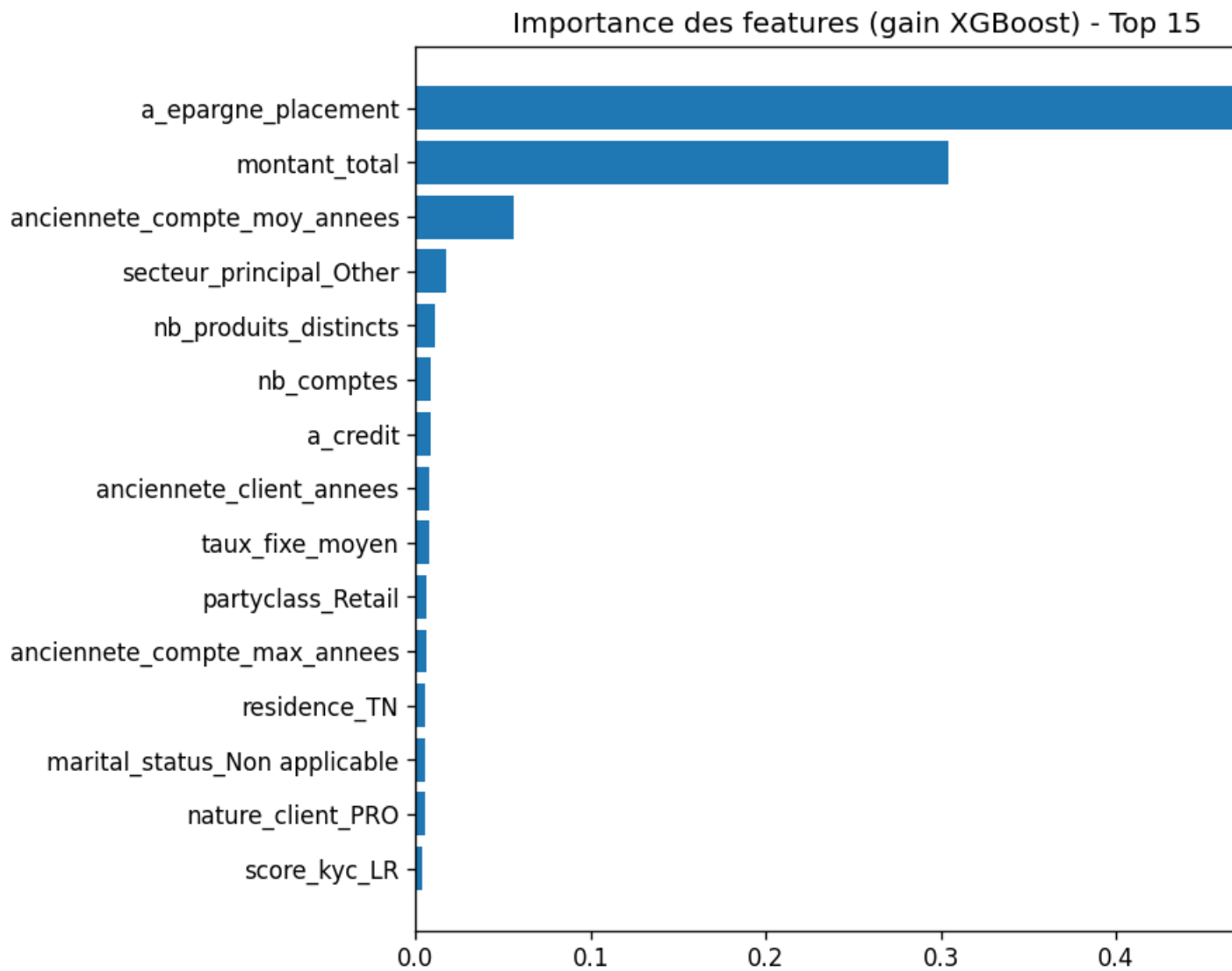


Fig. 6 : Importance des features (gain XGBoost)

4.14 6.7 Sauvegarde et intégration du modèle

Le modèle final, incluant l'intégralité du pipeline de prétraitement, est sérialisé avec joblib (04_machine_learning/models/model_final.joblib, environ 1 Mo). Ce choix garantit que l'inférence en production (application web, section 7) applique exactement les mêmes transformations qu'à l'entraînement, sans risque de divergence entre les deux environnements. Les versions de scikit-learn et XGBoost utilisées à l'entraînement sont épinglées dans les dépendances de l'application web, pour éviter toute incompatibilité de désérialisation du modèle liée à un changement de version. # 7. Interface web et déploiement

4.15 7.1 Objectifs et périmètre fonctionnel

L'application web a pour objectif d'exposer le modèle de churn et les analyses du projet à un utilisateur métier non technique, sans nécessiter d'installation ni de compétence en programmation. Conformément au périmètre minimal attendu, trois fonctionnalités ont été développées :

1. Des **KPIs de synthèse** sur la page d'accueil.
2. Une **analyse du churn par segment**, complémentaire du rapport Power BI.
3. Une **prédiction individuelle** (formulaire de saisie d'un profil client) et une **liste des comptes à risque** (prédiction en lot sur les clients actifs).

4.16 7.2 Choix technique

L'application est développée avec **Streamlit**, cohérent avec le choix de conserver Python comme langage unique sur l'ensemble de la chaîne (voir section 3.2). Ce choix a permis un développement rapide, l'équipe n'ayant pas eu à changer d'écosystème entre le pipeline ETL, la modélisation ML et l'interface de restitution.

Comme pour le pipeline ML, l'application interroge l'entrepôt PostgreSQL **en direct** à chaque affichage de page (pas de fichier CSV embarqué), avec une mise en cache des requêtes (10 minutes) pour limiter la charge sur la base sans sacrifier la fraîcheur des données affichées.

4.17 7.3 Page d'accueil — KPIs de synthèse

La page d'accueil affiche un état de santé du système (connexion à l'entrepôt, disponibilité du modèle), suivi des indicateurs clés : nombre de clients, nombre de comptes, taux de churn compte et taux de churn client, solde moyen, nombre de comptes clôturés, ainsi qu'une répartition rapide du portefeuille par statut et par secteur d'activité.

4.18 7.4 Analyse du churn par segment

Cette page reproduit, sous forme interactive et filtrable, les principaux axes de segmentation du churn déjà présents dans le rapport Power BI (âge, secteur d'activité, ancienneté), avec pour chaque segment le taux de churn et l'effectif associé affichés sous forme de graphique et de tableau détaillé.

4.19 7.5 Prédiction individuelle et comptes à risque

Cette page couvre les deux fonctionnalités centrales attendues :

Prédiction individuelle : deux modes de saisie sont proposés — la recherche d'un client existant par son identifiant (les features sont alors automatiquement récupérées

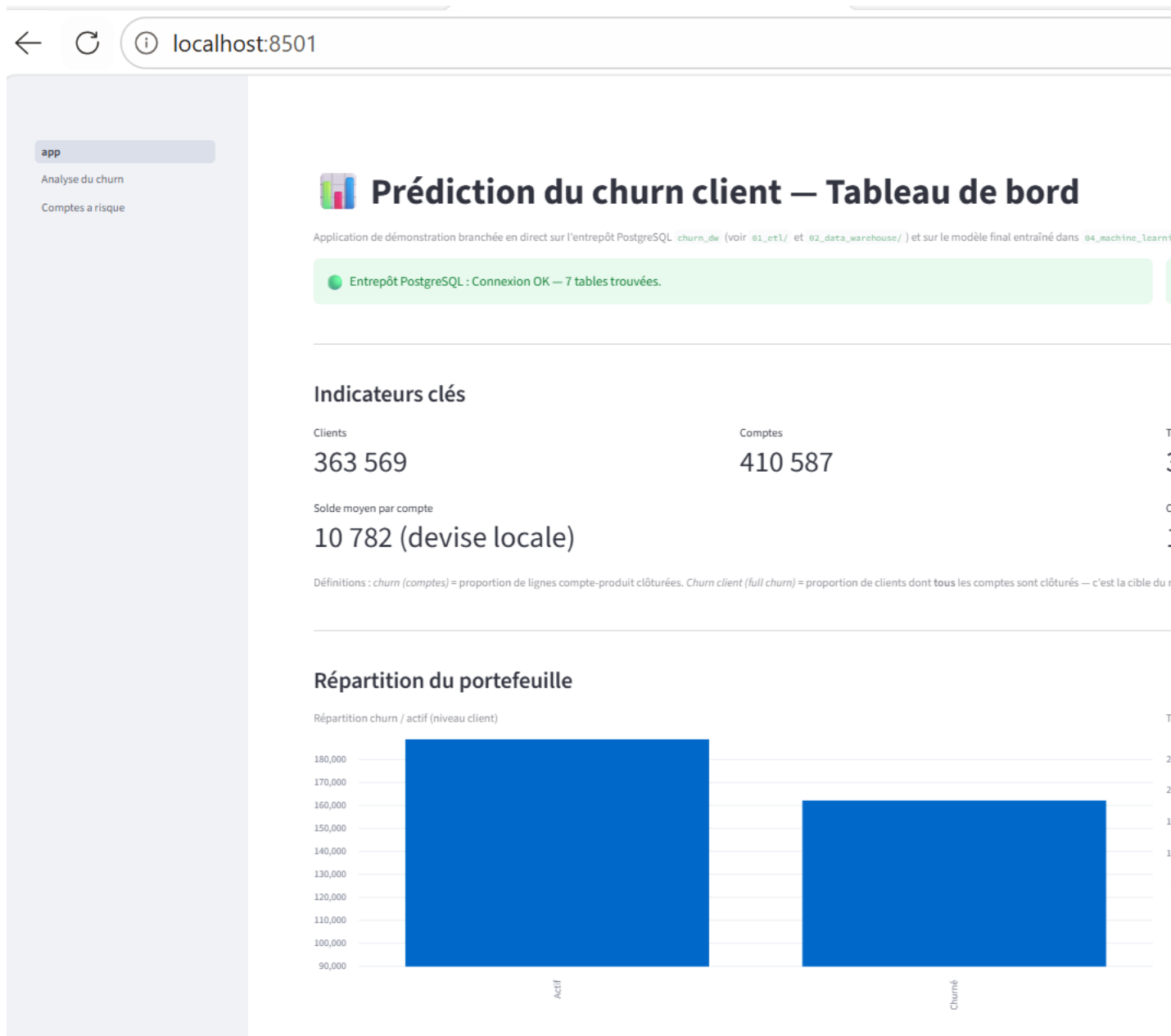


Fig. 7 : Page d'accueil de l'application web

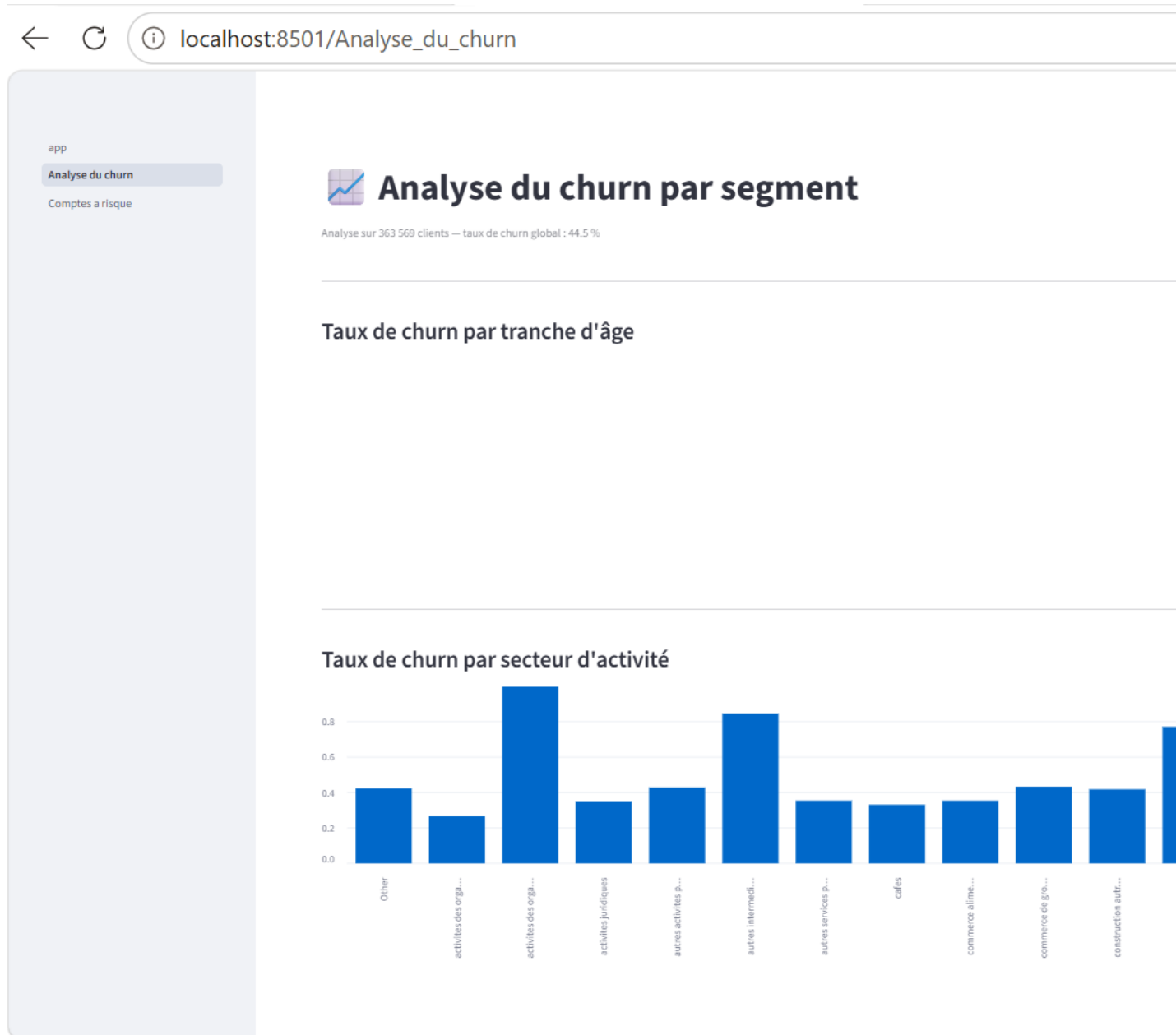


Fig. 8 : Page d'analyse du churn par segment

Fig. 9 : Formulaire de prédiction individuelle

depuis l'entrepôt), ou la saisie manuelle d'un profil complet, organisée par sections cohérentes avec l'analyse du modèle (profil socio-démographique, comportement bancaire, ancienneté, finances, produits détenus).

Liste des comptes à risque : une prédiction en lot est appliquée à l'ensemble des clients dont le dernier statut connu n'est pas déjà clôturé, avec un score de risque affiché pour chacun. La liste est filtrable par seuil de score et par secteur, triable par score décroissant, et exportable au format CSV pour une exploitation par les équipes de rétention.

4.20 7.6 Déploiement

Une procédure de déploiement sur Streamlit Community Cloud a été documentée (05_web_app/DEPLOY.md), incluant le point d'attention le plus critique : contrairement à une exécution locale, un déploiement cloud ne peut pas atteindre un PostgreSQL

hébergé sur la machine de développement — la procédure détaille la mise en place d'un PostgreSQL public gratuit (Neon) et le rechargement des données via le pipeline ETL vers cette instance distante, ainsi que la gestion des secrets de connexion. À la date de rédaction de ce rapport, l'application est fonctionnelle en environnement local ; le déploiement public final est en cours de finalisation (voir section 8, limites).

5. 8. Limites et perspectives

5.1 8.1 Limites méthodologiques

Définition du churn sans horizon temporel fixe. Le churn tel que modélisé est un churn *constaté* (le client a, à la date des données, tous ses comptes clôturés) et non un churn *prédictif à horizon fixe* (par exemple, “le client churmera dans les 90 prochains jours”). Cette seconde formulation, plus proche d’un cas d’usage opérationnel de rétention proactive, nécessiterait une date de référence glissante et un historique multi-période que le jeu de données à disposition, en coupe unique, ne permet pas de construire.

Valeurs manquantes structurelles sur l’ancienneté des comptes. Les variables ANCIENNETE_COMPTE_MOY_ANNEES et ANCIENNETE_COMPTE_MAX_ANNEES sont manquantes pour environ 86 % des clients churnés contre 0 % des clients actifs, du fait du filtrage des dates d’ouverture antérieures à 1900 (section 4.1.2). Il ne s’agit pas d’une fuite de données au sens strict (l’imputation par la médiane ne transmet pas d’indicateur de valeur manquante au modèle), mais ce déséquilibre de complétude entre les deux classes doit être gardé à l’esprit dans l’interprétation des résultats, et pourrait justifier un travail de fiabilisation des dates anciennes côté système source.

Catégories de compte majoritairement non renseignées. Comme détaillé en section 5.4, 54,9 % des comptes n’ont pas de catégorie renseignée dans les données sources. Le taux de churn élevé associé à cette catégorie « Non renseigné » (58,2 %) est vraisemblablement le reflet d’une qualité de donnée dégradée plutôt qu’un vrai signal de risque métier — un point à ne pas sur-interpréter en l’état.

Secteurs d’activité à faible effectif. Certains secteurs d’activité affichant les taux de churn les plus extrêmes (proches de 100 %) ne comptent que quelques centaines de comptes. Ces résultats, bien que réels dans les données, doivent être interprétés avec prudence et ne constituent pas nécessairement un signal généralisable, contrairement au résultat sur la détention d’épargne (section 6.6), retrouvé de façon convergente sur l’ensemble du portefeuille par deux méthodes indépendantes.

5.2 8.2 Limites d’ingénierie

Rapport Power BI partiellement finalisé. Comme indiqué section 5.3, une page combinant vue d’ensemble et analyse du churn est opérationnelle ; la page dédiée à la segmentation client (profils, valeur, comptes à risque) reste à construire, bien que sa structure et ses mesures DAX soient d’ores et déjà documentées.

Déploiement public de l’application web en cours de finalisation. L’application est pleinement fonctionnelle en environnement local, connectée à l’entrepôt PostgreSQL local. Le déploiement sur Streamlit Community Cloud nécessite la mise en place d’un PostgreSQL accessible publiquement (section 7.6), étape en cours au moment de la rédaction de ce rapport.

Absence d’optimisation des hyperparamètres. Les modèles ont été entraînés avec des hyperparamètres raisonnables mais non systématiquement optimisés (pas de recherche en grille ou bayésienne). Un gain de performance supplémentaire, probablement marginal compte tenu des scores déjà élevés (PR-AUC 0,983), pourrait être obtenu par un réglage plus poussé du modèle XGBoost retenu.

5.3 8.3 Perspectives d'amélioration

- **Churn prédictif à horizon fixe** : si un historique multi-période devenait disponible, reformuler la cible en probabilité de churn à N jours plutôt qu'un constat de statut actuel, pour un usage opérationnel de rétention proactive plus direct.
- **Enrichissement transactionnel** : le référentiel de transactions mis à disposition (`dim_TRANSACTION`) n'a pas été exploité dans la version actuelle du pipeline ; l'intégration de features comportementales transactionnelles (fréquence, régularité des mouvements) constituerait une piste naturelle d'amélioration du pouvoir prédictif du modèle.
- **Suivi de la dérive du modèle** : en cas de mise en production réelle, mettre en place un suivi de la performance du modèle dans le temps (data drift, concept drift), le comportement client et les conditions macroéconomiques pouvant évoluer.
- **Étendre l'analyse SHAP** à des visualisations d'interaction entre variables (SHAP dependence plots), pour affiner la compréhension des effets croisés entre ancienneté, produits détenus et secteur d'activité.
- **Finaliser la page de segmentation Power BI** et le déploiement public de l'application web, les deux actions restant à court terme les plus directement actionnables pour compléter le périmètre du projet.

6. 9. Conclusion

Ce projet a permis de construire, de bout en bout, une chaîne décisionnelle complète pour l'analyse et la prédiction du churn client : de l'extraction et du nettoyage des données brutes jusqu'à une application web opérationnelle exposant un modèle prédictif, en passant par un entrepôt de données structuré et un rapport d'analyse Business Intelligence.

Le résultat le plus robuste du projet — la détention d'un produit d'épargne ou de placement comme facteur fortement protecteur contre le churn — a émergé de façon convergente par deux voies d'analyse indépendantes (analyse descriptive SQL/Power BI, interprétabilité SHAP du modèle prédictif), ce qui en fait une recommandation métier particulièrement défendable pour des actions de rétention ciblées.

Au-delà des résultats obtenus, le projet a été l'occasion d'exercer une rigueur méthodologique explicite : deux fuites de données ont été identifiées, investiguées par un test de sensibilité chiffré, documentées et corrigées plutôt que traitées superficiellement ; les limites et zones d'incertitude des données (complétude, effectifs faibles sur certains segments) sont rapportées avec la même transparence que les résultats positifs.

Les axes restant à finaliser — la page de segmentation Power BI et le déploiement public de l'application web — sont des extensions incrémentales d'une architecture déjà fonctionnelle de bout en bout, et ne remettent pas en cause la validité des analyses et du modèle prédictif présentés dans ce rapport.

7. Annexes

7.1 A. Glossaire

Terme	Définition
Churn	Attrition, perte d'un client (ici : clôture de la totalité de ses comptes)
ETL	Extract-Transform-Load, pipeline d'intégration de données
Grain	Niveau de détail d'une table de faits (ici : compte-produit ou client selon le contexte)
KYC	<i>Know Your Customer</i> , processus de connaissance client réglementaire
PR-AUC	Aire sous la courbe précision-rappel, métrique adaptée aux classes déséquilibrées
ROC-AUC	Aire sous la courbe ROC (<i>Receiver Operating Characteristic</i>)
SHAP	<i>SHapley Additive exPlanations</i> , méthode d'interprétabilité des modèles ML
Surrogate key (SK)	Clé de substitution technique, utilisée dans les schémas dimensionnels

7.2 B. Structure du dépôt de code

Le code source complet est disponible sur le dépôt GitHub du projet : <https://github.com/issraague>

Dossier	Contenu
00_documentation/	Documents de cadrage fournis par l'encadrant
01_etl/	Pipeline d'extraction, nettoyage et transformation des données
02_data_warehouse/	Schéma SQL, scripts de chargement, documentation des KPIs
03_power_bi/	Guide de construction et mesures DAX du rapport Power BI
04_machine_learning/	Notebooks et scripts d'entraînement, comparaison des modèles, modèle final
05_web_app/	Application Streamlit et procédure de déploiement
06_rapport/	Le présent rapport (sources Markdown/LaTeX et version PDF)
07_presentation/	Slides de présentation

Chaque dossier dispose d'un `README.md` détaillant les étapes d'exécution et les éven-

tuels prérequis, pour permettre à un tiers de reproduire l'intégralité du pipeline.

7.3 C. Références des outils utilisés

- **Python 3.13**, **pandas**, **SQLAlchemy**, **psycopg2** pour l'ETL et l'accès aux données.
- **PostgreSQL 16** comme entrepôt de données analytique.
- **Power BI Desktop** pour les tableaux de bord.
- **scikit-learn**, **XGBoost**, **SHAP** pour la modélisation et l'interprétabilité.
- **Streamlit** pour l'application web.
- **Git** et **GitHub** pour le versioning et la collaboration.